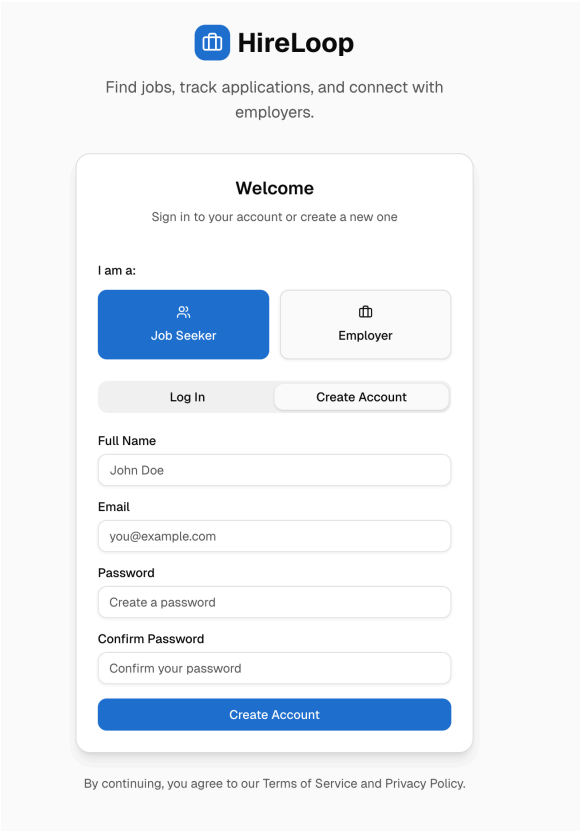
Group Members: Kevin Dong, Katelyn Doucette, Avi Maheshwari, Eshaan Salvi, Charan Sama, Rishi Chawla

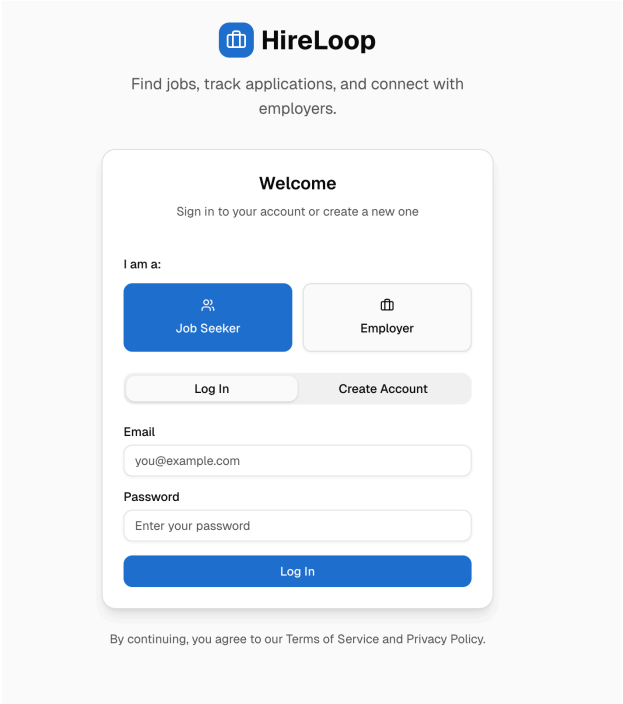
Github link: https://github.com/RishiChawla1/CS3704-HireLoop

UI Design or Mock UIs

Screen 1 — Authentication / Landing

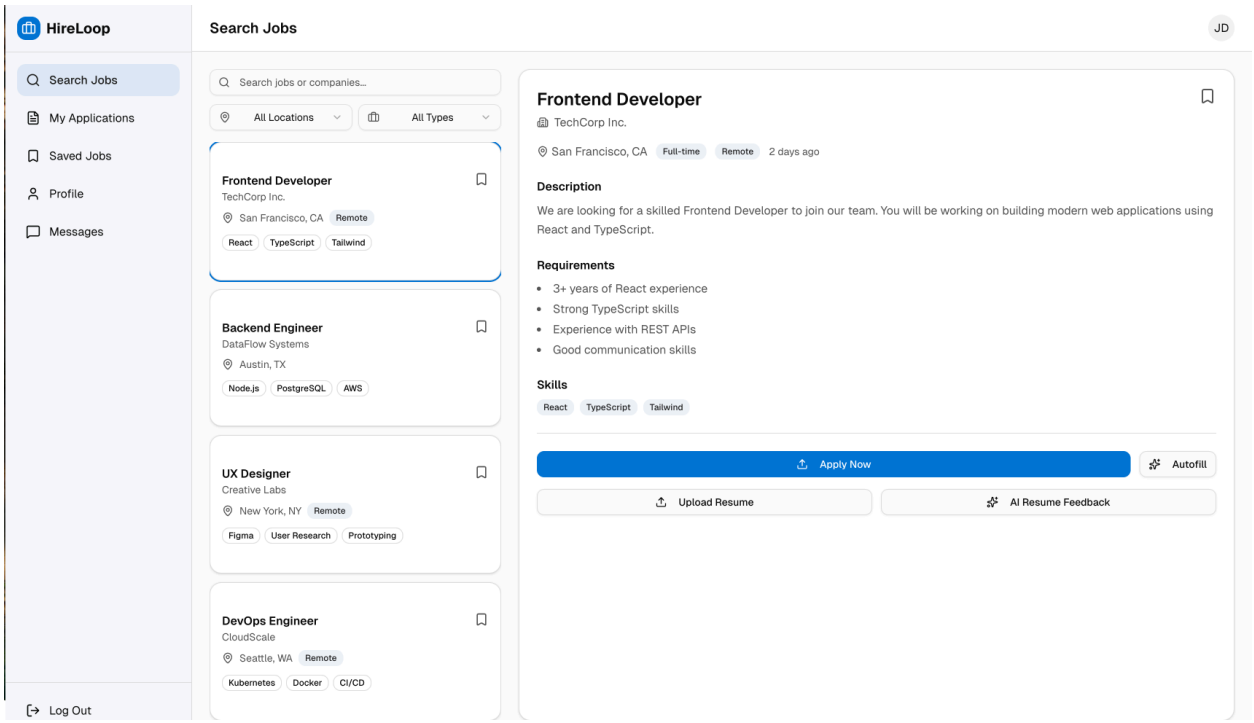
- UUC5: Sign up page, create account
- AUC7: User logs in





Screen 2 — Candidate Main Dashboard

- UUC1: Search for job posting
- UUC2: View job posting details
- UUC3: Application statuses
- UUC6: Apply to a job posting
- UUC7: Filter job search results
- UUC8: Edit profile for users
- DUC3: User information autofill on application
- DUC4: Track application status
- DUC6: Store bookmarked job postings
- AUC2: Get AI feedback on resume
- AUC4: Submit a job application
- AUC5: Get status of application
- AUC6: Update applicant profile picture



HireLoop

Search Jobs
My Applications
Saved Jobs
Profile
Messages

My Applications

Application Status
Track the progress of your job applications

Frontend Developer TechCorp Inc.	Jan 15, 2024	Interview
Full Stack Developer StartupXYZ	Jan 10, 2024	Applied
React Developer WebAgency	Jan 5, 2024	Rejected
Software Engineer BigTech Co.	Dec 20, 2023	Accepted

HireLoop

Search Jobs
My Applications
Saved Jobs
Profile
Messages

My Profile

My Profile
Manage your profile information

JD
Upload Photo

Full Name
John Doe

Email
john.doe@email.com

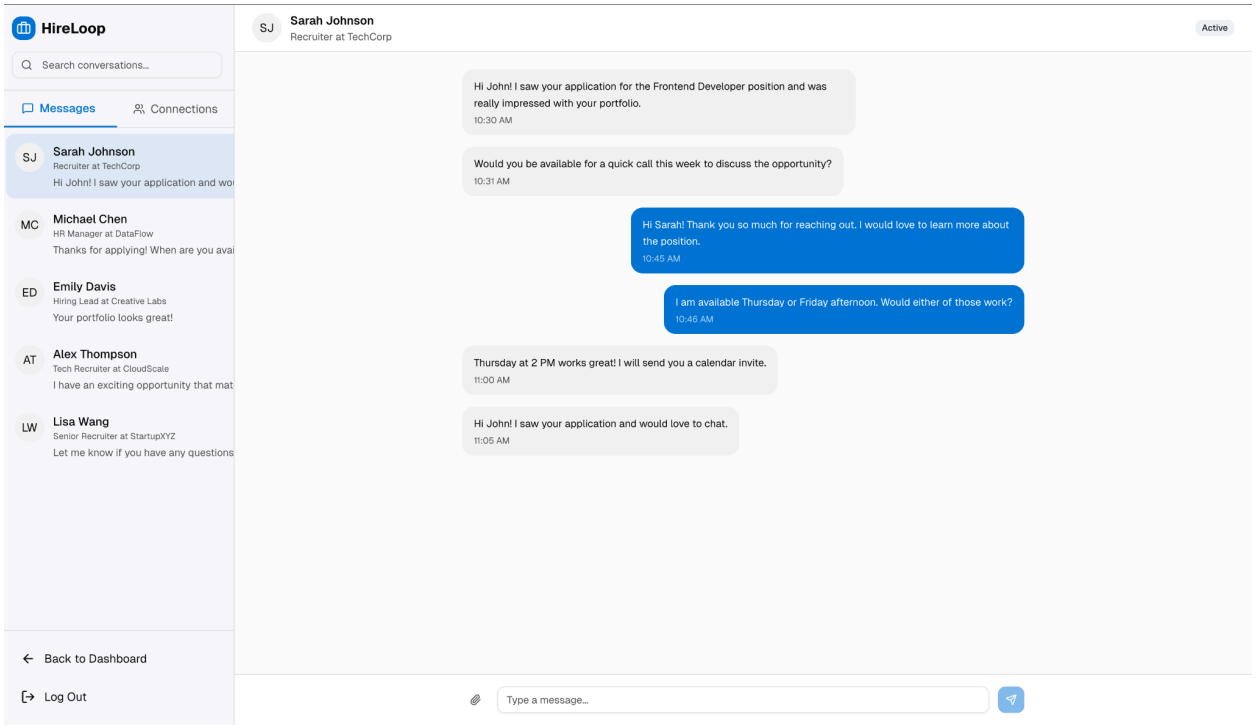
Skills
React TypeScript Node.js PostgreSQL + Add Skill

Resume
resume_john_doe.pdf
Upload

Save Changes

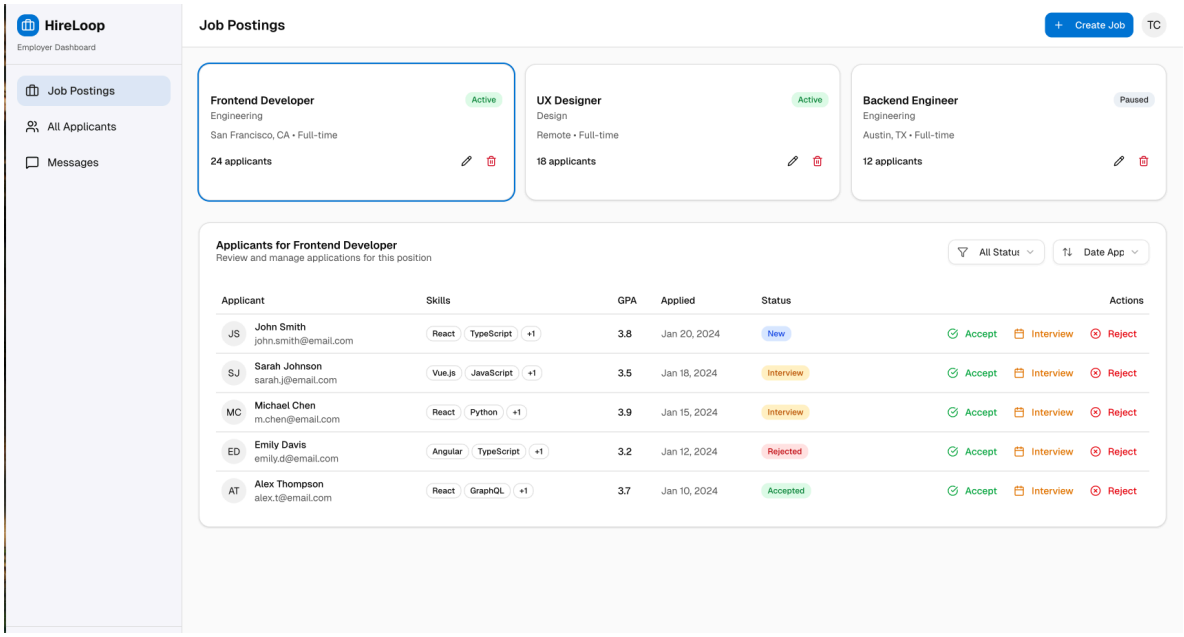
Screen 3 — Messaging / Connections

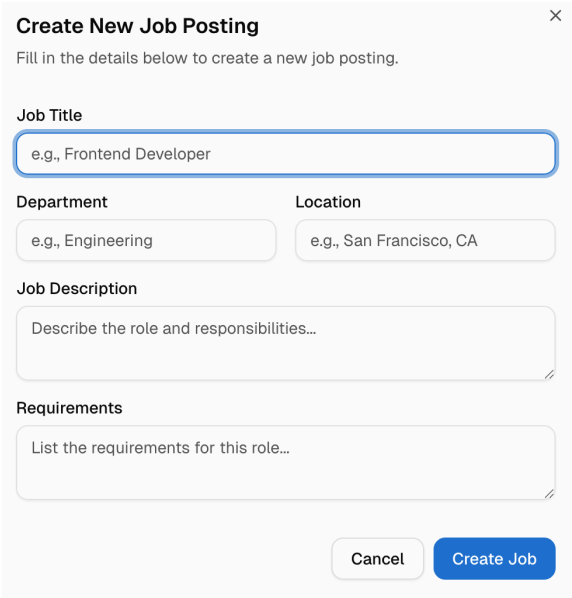
- UUC4: Direct messaging
- DUC5: Store messages between users
- DUC7: Store user connections



Screen 4 — Employer Dashboard

- AUC1: Create job opening
- AUC3: Edit job opening
- AUC8: Delete job posting
- DUC1: Send response to application
- DUC2: Organize applicants by tags
- DUC8: Provide list of users based on tags





Algorithm Design expressed with pseudo-code

Algorithm 1: The **findNearestFitJob()** function

```
Input: Applicant identifier
Output: The best fitting job for that applicant
int score;
load skills, demographic data, education, and work history into one
  form;
call to database: return the jobs with the right restrictions and
  specification to match user data;
while (there are still jobs) do
    score = matchCriteriaToScore()
    if score > bestSeenScore then
        bestJob = currentJob
        bestSeenScore = score
return bestJob
```

Algorithm 2: The **matchCriteriaToScore()** function

```
Input: The list of weights computed from an applicant
Output: The final score for how good a candidate is for a job match
int skillWeight = a;
int demographicWeight = b;
int educationWeight = c;
int workHistoryWeight = d;
weights = [a,b,c,d];
foreach Weight to consider do
    Increase score proportional to similarity
return score;
```

Algorithm 3: The **matchApplicantsToJobs()** function

```
Input: The list of applicants, and list of jobs as well as their criteria
Output: A one-to-one map between best jobs and applicants
Create applicant and job opening nodes;
foreach user and job opening do
    assign them to one bipartite graph node.
    use matchCriteriaToScore() to determine weights between jobs
      and applicants.
    assign that weight to the node.
while There are still job openings do
    assign matches between one applicant and one job opening based on
      nearest fit job;
    remove them from the graph and notify applicant and employer of
      match;
```

Algorithm 4: The **requestAIReview()** function

```
Input: The applicant resume
Output: The output of the resume on success, or an error on failure.
// Validate the pdf using an external library
if pdf is not valid then
    return error
construct request to external AI service, append application.
Send request and wait
if connection was successful then
    Parse out the AI response, return it to the user
else
    return error
```

Algorithm 5: The **filterJobsByName()** function

```
Input: the list of jobs to filter, and search term
Output: the list of jobs that are within some set threshold for
  searching (tuneable)
foreach job name do
    /* Existing code is out there to compute the levenstein distance
      between two strings */
    l = levenstein distance to the job name
    if l ¡ threshold then
        append to return list
return final list
```

Algorithm 6: The **validateUserProfileUpdate()** function

```
Input: Takes in a user, a list of field updates, and the current browser
       session cookie
Output: outputs whether or not the update is legal
if cookie is not legal then
    return false
foreach Updated field do
    if Field length is too long, or contains banned words or characters
      then
        return false
return true
```

Algorithm 7: The **closeJobOpening()** function

```
Input: Takes in the user cookie, and the job opening
Output: Whether it successfully closed the job opening
if Job opening is still open then
    return false
if User cookie is invalid then
    return false
create connection to database
if connection request failed then
    return false
request list of applicants for job opening from database
foreach applicant do
    if Candidate not marked as chosen then
        Set "accepted" field for this job to false
Return updated data to database
close connection to database
return true;
```